

assemble_ab_disk

assemble_ab_disk.sh — user guide

Revision: 1 **Last modified:** 2026-06-11T00:00:00Z

Companion documentation (§11.4.18) for tests/emulator/ab_virt/
assemble_ab_disk.sh.

Overview

assemble_ab_disk.sh takes the built guest artifacts (kernel Image + rootfs.ext2 + u-boot.bin) and assembles a deterministic **2-slot (A/B) GPT disk image** ab_disk.img plus a compiled U-Boot script boot.scr. The intent is that the U-Boot A/B boot script (uboot_ab/boot.cmd → boot.scr) selects between and rolls back across the two root slots, so PWU-AB-1 can boot the disk under QEMU -machine virt + HVF and demonstrate a slot switch.

GPT layout the script produces:

Part	Type	Label	Role
p1	FAT32	boot	kernel Image + boot.scr (U-Boot virtio 0:1)
p2	ext2/4	rootfs_a	slot A root, seeded from rootfs.ext2, tagged /etc/slot_id=A → /dev/vda2
p3	ext2/4	rootfs_b	slot B root, distinct copy, tagged /etc/slot_id=B → /dev/vda3

The /etc/slot_id marker per slot is what makes a slot switch *observable* from inside the guest (findmnt / + cat /etc/slot_id).

Status — AUTHORED, NOT YET RUN (UNVERIFIED, pending u-boot.bin). This script has been authored but has **not** been executed end-to-end. It hard-requires a real out/images/u-boot.bin (it aborts with exit 3 if

absent), and that bootloader was still building at authoring time. The actual **A/B slot switch is therefore UNPROVEN**: there is no captured evidence yet that U-Boot selects the correct slot, that the `/etc/slot_id` markers are read as intended, or that rollback works. No claim is made here that the slot switch functions — that is pending a real run against a produced `u-boot.bin`. (§11.4.6 — documented as unconfirmed, not asserted.)

Prerequisites

- `podman` on `PATH` (the only hard host dependency; aborts with exit 3 if absent).
- The inputs below, produced by a prior `build_image.sh` run — including `u-boot.bin`, which is the gating input that is not yet available.
- `uboot_ab/boot.cmd` present (the U-Boot script source compiled to `boot.scr`).

Usage examples

```
# Assemble the 2-slot GPT disk (requires u-boot.bin to exist).
tests/emulator/ab_virt/assemble_ab_disk.sh

# Also wipe a prior disk image + stale assemble container first.
tests/emulator/ab_virt/assemble_ab_disk.sh --clean

# Override disk + boot-partition sizing.
HELIX_AB_DISK_MB=1024 HELIX_AB_BOOT_MB=64 \
  tests/emulator/ab_virt/assemble_ab_disk.sh
```

Inputs

Input	Source	Default / requirement
<code>--clean</code>	CLI arg 1	(none)
<code>HELIX_AB_DISK_MB</code>	env	1024 (auto-bumped if below the needed size)
<code>HELIX_AB_BOOT_MB</code>	env	64
<code>out/images/Image</code>	filesystem (from <code>build_image.sh</code>)	required, non-empty
<code>out/images/rootfs.ext2</code>	filesystem	required, non-empty
<code>out/images/u-boot.bin</code>	filesystem	required, non-empty — the gating input
<code>uboot_ab/boot.cmd</code>	filesystem	required

Fixed object name: the named assemble container `helix_ab_assemble`.

Outputs

All under `tests/emulator/ab_virt/out/`, gitignored (§11.4.30; this script is the §11.4.77 regeneration mechanism):

Output	Meaning
<code>out/images/ab_disk.img</code>	the 2-slot GPT disk
<code>out/images/boot.scr</code>	compiled U-Boot boot script
<code>out/assemble.log</code>	full assembly transcript
<code>out/.disk_ok</code>	success stamp — written only when <code>ab_disk.img</code> exists non-empty

Side-effects

- Starts a detached named podman container `helix_ab_assemble` (`sleep 600`), copies inputs in, runs the assembly, copies outputs out.
- A `trap cleanup EXIT` force-removes the container on every exit path.
- `--clean` additionally removes the prior `ab_disk.img`, `boot.scr`, and `.disk_ok`.
- No `/Volumes` bind-mount and no `:Z` SELinux relabel (both rejected by macOS podman); all I/O is via `podman cp`.

Internal behaviour

1. `set -u + set -o pipefail`; resolve paths, sizing, and the container name.
2. Abort (exit 3) if podman absent; abort (exit 3) if any required input (`Image`, `rootfs.ext2`, `u-boot.bin`, `boot.cmd`) is missing/empty.
3. On `--clean`, remove the prior disk/script/stamp + a stale container.
4. Remove any leftover `out/.disk_ok`; `mkdir -p out/images`.
5. `podman run -d` the named assemble container; install `trap cleanup EXIT`.
6. Copy `Image`, `rootfs.ext2`, `u-boot.bin`, `boot.cmd` into `/asm/in`.
7. `podman exec` the in-container assembly (an ASCII-safe single-quoted bash `-c` string — no apostrophes/parens, a lexical rule that broke earlier builds):
 - `apt-get install gdisk dosfstools e2fsprogs mtools u-boot-tools fdisk util-linux`;
 - `mkimage compile boot.cmd → boot.scr` (fail with `ASM_FAIL` / exit 11 if not produced);
 - compute slot sizing from the `rootfs` size + 32 MiB headroom; bump the requested disk size if too small;
 - truncate a raw disk and lay down a deterministic GPT (`sgdisk`): `p1 boot FAT (0700)`, `p2 rootfs_a (8300)`, `p3 rootfs_b (8300)`;
 - read each partition's start/size in 512-byte sectors via `sfdisk -d` (fail with `ASM_FAIL` / exit 12 if offsets unreadable);
 - build the FAT boot partition (`mkfs.fat -F 32`, `mcopy` the kernel + `boot.scr`);
 - prepare slot A (copy `rootfs`, grow to partition size, `e2fsck + resize2fs`, write `/etc/slot_id=A` via `debugfs`);
 - prepare slot B (distinct copy, same treatment, `/etc/slot_id=B`);

- `dd` the three partition images into the GPT disk at their sector offsets; `sync`;
print the final GPT and `ASM_OK`.
- 8. Treat the run as failed if the `exec rc` is non-zero **or** `ASM_OK` is not present in
`assemble.log`; on failure log the tail and exit 1.
- 9. Copy `ab_disk.img` + `boot.scr` out via `podman cp`.
- 10. **Anti-bluff gate (§11.4.6)**: stamp out `./disk_ok` and exit 0 **only** when
`ab_disk.img` exists non-empty; otherwise exit 1 without stamping.

Edge cases

- **podman absent / any required input missing** → exit 3 before assembly. In particular, a not-yet-built `u-boot.bin` aborts the script — this is the current blocking condition.
- **boot.scr not produced by mkimage** → `ASM_FAIL` exit 11.
- **GPT offsets unreadable** → `ASM_FAIL` exit 12.
- **Requested disk size too small** → auto-bumped to the computed needed size (logged `ASM_NOTE`).
- **debugfs cannot overwrite /etc/slot_id** → falls back to a plain write (handles the case where the file does not pre-exist).
- **Assembly finishes but ab_disk.img is missing/empty** → exit 1, `./disk_ok` not stamped.
- **Slot switch itself** → UNVERIFIED (see the status note above); no positive evidence exists yet, and none is claimed.

Related scripts

- `tests/emulator/ab_virt/build_image.sh` — produces the Image + `rootfs.ext2` + `u-boot.bin` this script consumes. See `docs/scripts/build_image.md`.
- `tests/emulator/ab_virt/boot_smoke.sh` — base-image boot smoke (the proven foundation). See `docs/scripts/boot_smoke.md`.
- `uboot_ab/boot.cmd` + `uboot_ab/README.md` — the U-Boot A/B coherence contract this disk layout must match.
- `docs/research/rk3588_emulator/REPORT.md` — the emulator design report (§3/§4, PWU-AB-1).

Last verified date

2026-06-11 — documented against the script as committed (read, not inferred). The script is **AUTHORED** but **NOT YET RUN**; the A/B slot switch is **UNVERIFIED** pending a real `u-boot.bin`.